DATABASE MANAGEMENT SYSTEMS
ASSIGNMENT – CO2

Relational Design and Validation for Courier Handover Completion

SwiftSpan Parcels | Assignment 1

Student Name: Preethi G

Register Number: 1924260136

TASK 1 – Design Alternative Relational Schemas

The universal observation relation contains 14 attributes: SenderID, SenderName, RouteGroup, ServiceID, ServiceCode, ServiceName, VolumeWeight, CoordinatorID, CoordinatorName, Hub, DispatchWave, Completion, ServiceBand and RouteFactor. The logical observation key is (SenderID, ServiceID, DispatchWave).

Design A: Flat baseline

```
HANDOVER_FLAT(SenderID, SenderName, RouteGroup, ServiceID, ServiceCode, ServiceName, VolumeWeight,
CoordinatorID, CoordinatorName, Hub, DispatchWave, Completion, ServiceBand, RouteFactor)
PK (SenderID, ServiceID, DispatchWave)
UNIQUE (SenderID, ServiceCode, DispatchWave)
```

This design stores every fact on each observation row. It is simple for a single-row read but repeats sender, service, coordinator and policy facts. It is the baseline for measuring update anomalies and storage.

Design B: Partial decomposition

```
SENDER(SenderID PK, SenderName, RouteGroup)
SERVICE(ServiceID PK, ServiceCode UNIQUE, ServiceName, VolumeWeight)
HANDOVER_PARTIAL(SenderID, ServiceID, DispatchWave, CoordinatorID, CoordinatorName, Hub, Completion,
ServiceBand, RouteFactor)
PK HANDOVER_PARTIAL(SenderID, ServiceID, DispatchWave)
FK SenderID -> SENDER; FK ServiceID -> SERVICE
```

This removes sender identity and service catalog repetitions. It deliberately retains CoordinatorName and Hub in each handover row even though CoordinatorID determines them. It also retains ServiceBand and RouteFactor with Completion. Thus it is a meaningful intermediate design, not a fully normalized design.

Design C: BCNF decomposition

```
SENDER(SenderID PK, SenderName, RouteGroup)
SERVICE(ServiceID PK, ServiceCode UNIQUE, ServiceName, VolumeWeight)
COORDINATOR(CoordinatorID PK, CoordinatorName, Hub)
ASSIGNMENT(SenderID, ServiceID, CoordinatorID, PK(SenderID,ServiceID), FK SenderID, FK ServiceID, FK
CoordinatorID)
COMPLETION_POLICY(Completion PK, ServiceBand)
SERVICE_BAND(ServiceBand PK, RouteFactor)
OBSERVATION(SenderID, ServiceID, DispatchWave, Completion,
            PK(SenderID,ServiceID,DispatchWave),
            FK(SenderID,ServiceID)->ASSIGNMENT,
            FK Completion->COMPLETION_POLICY)
```

SERVICE has an alternate candidate key ServiceCode because it is unique and identifies the same catalog object as ServiceID. The observation references ASSIGNMENT so that only a valid sender/service assignment can be observed. COMPLETION_POLICY represents the fixed completion-to-band mapping; SERVICE_BAND represents the band-to-route-factor mapping.

The determinant-based links are: SenderID to SENDER; ServiceID or ServiceCode to SERVICE; CoordinatorID to COORDINATOR; (SenderID,ServiceID) to ASSIGNMENT; Completion to COMPLETION_POLICY; ServiceBand to SERVICE_BAND; and the three-part observation key to OBSERVATION.

The flat design is worth testing for low join count; the partial design tests the benefit of removing identity/catalog repetition while leaving assignment and policy repetition; the BCNF design tests minimum repeated facts and lower correction scope against the cost of additional joins.

TASK 2 – Functional Dependencies

The accepted dependencies follow from the business rules stated in the question, not from patterns in sample data:

• SenderID → SenderName, RouteGroup (a sender has one identity and route group).

• ServiceID → ServiceCode, ServiceName, VolumeWeight (a service ID identifies one catalog object).

• ServiceCode → ServiceID (ServiceCode is unique and identifies the same catalog object).

• (SenderID, ServiceID) → CoordinatorID (the coordinator is assigned to a sender/service pair across dispatch waves).

• CoordinatorID → CoordinatorName, Hub (a coordinator has one name and hub).

• (SenderID, ServiceID, DispatchWave) → Completion (the full observation key determines completion).

• Completion → ServiceBand and ServiceBand → RouteFactor (the fixed fictional policy defines these mappings).

Relative to K = {SenderID, ServiceID, DispatchWave}, the key-to-Completion dependency is full: no proper subset of K determines Completion. SenderID → SenderName, RouteGroup and ServiceID → service catalog facts are partial dependencies because they depend on proper subsets of the composite key. The dependencies from CoordinatorID, Completion and ServiceBand are transitive consequences from the key through non-key determinants.

Do not accept RouteGroup → SenderID or a dependency involving one particular DispatchWave merely because generated cohorts happen to exhibit it. Such a pattern is not guaranteed by the stated business rules.

TASK 3 – Closures, Candidate Keys and Minimal Cover

Closure of the primary observation key

```
{SenderID, ServiceID, DispatchWave}+
1. SenderID -> SenderName, RouteGroup
2. ServiceID -> ServiceCode, ServiceName, VolumeWeight
3. SenderID, ServiceID -> CoordinatorID
4. CoordinatorID -> CoordinatorName, Hub
5. SenderID, ServiceID, DispatchWave -> Completion
6. Completion -> ServiceBand
7. ServiceBand -> RouteFactor
Closure = all 14 attributes.
```

Closure using the alternate business identifier

```
{SenderID, ServiceCode, DispatchWave}+
1. ServiceCode -> ServiceID
2. Apply the preceding closure using SenderID, ServiceID, DispatchWave
Closure = all 14 attributes.
```

Every candidate key must contain SenderID and DispatchWave because neither is determined by any stated dependency. It must also contain either ServiceID or ServiceCode to identify the service. The two minimal candidate keys are therefore (SenderID,

ServiceID, DispatchWave) and (SenderID, ServiceCode, DispatchWave). Removing any one attribute from either set loses sender identity, service identity, or the dispatch period, so each proper subset fails.

Prime attributes: SenderID, ServiceID, ServiceCode and DispatchWave. Non-prime attributes: SenderName, RouteGroup, ServiceName, VolumeWeight, CoordinatorID, CoordinatorName, Hub, Completion, ServiceBand and RouteFactor.

Minimal cover

```
SenderID -> SenderName
SenderID -> RouteGroup
ServiceID -> ServiceCode
ServiceID -> ServiceName
ServiceID -> VolumeWeight
ServiceCode -> ServiceID
SenderID, ServiceID -> CoordinatorID
CoordinatorID -> CoordinatorName
CoordinatorID -> Hub
SenderID, ServiceID, DispatchWave -> Completion
Completion -> ServiceBand
ServiceBand -> RouteFactor
```

Each right-hand side is a single attribute; no left-hand attribute is extraneous in the composite determinants; and no listed dependency is redundant under the remaining set. In particular, the apparent direct dependency (SenderID,ServiceID) → CoordinatorName is redundant because it follows through CoordinatorID. The catalog attribute VolumeWeight remains determined by ServiceID; if it appears redundantly on an augmented dependency whose left side already determines ServiceID and whose right side includes VolumeWeight, remove that redundant right-side consequence from the minimal cover and retain ServiceID → VolumeWeight.

The original set implies the cover because each cover dependency is a stated rule or a single-attribute consequence of an augmented rule. The cover implies the original because the omitted scope-to-name consequence follows by transitivity: (SenderID,ServiceID) → CoordinatorID and CoordinatorID → CoordinatorName. Thus both sets imply one another.

TASK 4 – Update, Insertion and Deletion Anomalies

Insertion anomaly: in the flat relation, a new service catalog item cannot be stored without inventing a sender/dispatch observation. In the partial design, SERVICE allows an independent catalog insertion. In the BCNF design, SERVICE and policy facts can be inserted independently.

Deletion anomaly: deleting the last observation for a service from the flat relation can also delete the only stored copy of ServiceName and VolumeWeight. In the partial design, the separate SERVICE row survives deletion of the final HANDOVER_PARTIAL row. In the BCNF design, catalog facts are independent of observations.

Update anomaly: changing ServiceName in the flat table requires updating every observation for that ServiceID; missed rows create inconsistent copies. The partial and BCNF designs update SERVICE once. The partial design still repeats CoordinatorName/Hub across dispatch waves and repeats ServiceBand/RouteFactor across observations. The BCNF design stores those facts once in COORDINATOR and policy tables.

For a change to ServiceID=1, the flat design touches every observation row with ServiceID=1. The partial design updates one SERVICE row for catalog facts, while the BCNF design also updates one SERVICE row. The question supplies total observation counts (10,000; 100,000; 600,000) but not the generated distribution of ServiceID=1, so exact affected-row counts cannot be calculated from the paper alone. Report the measured count from the generated dataset rather than assuming uniformity.

Allocated bytes must be measured from the actual MySQL tables and indexes. Any hand-calculated row-width estimate must be labeled an estimate and must state datatype, nullability, index and page-overhead assumptions.

TASK 5 – Normal Forms up to BCNF

The flat relation is in 1NF if all fields are atomic. It violates 2NF because non-prime attributes such as SenderName and ServiceName depend on proper subsets of the composite candidate key. It also violates 3NF/BCNF through determinants such as CoordinatorID, Completion and ServiceBand that are not superkeys.

SENDER, SERVICE and COORDINATOR are in BCNF because their determinants are candidate keys. ASSIGNMENT is in BCNF under the declared dependency (SenderID,ServiceID) → CoordinatorID. COMPLETION_POLICY and SERVICE_BAND are in BCNF because Completion and ServiceBand respectively are keys. OBSERVATION is in BCNF when its only nontrivial projected dependency is the full key → Completion; the foreign key to ASSIGNMENT is an inclusion constraint, not an additional FD.

The synthesis groups are the determinant groups listed in the minimal cover. The BCNF binary decomposition separates each violating determinant into a relation containing determinant and dependent attributes, while retaining the determinant as a key in that relation. The design in Task 1 is the resulting determinant-based schema.

For the separate example AB → D and D → B, candidate key AB determines D and then B. D → B violates BCNF because D is not a superkey. The relation can satisfy 3NF if B is prime (B belongs to a candidate key such as DB), while failing BCNF. A BCNF split may lose dependency preservation for AB → D, illustrating the trade-off.

TASK 6 – Lossless Join, Dependency Preservation, 4NF and 5NF

The decomposition is lossless when each binary split uses a common attribute set that functionally determines one of the two component relations. For example, splitting sender facts from the universal relation uses SenderID, which determines SenderName and RouteGroup; splitting service facts uses ServiceID, which determines the catalog attributes. The assignment split uses (SenderID,ServiceID), which determines CoordinatorID. These determinant-based splits provide lossless joins when the relations are joined on their declared keys.

A split that separates facts merely on common ServiceID can be lossy if the two sides contain independent sender/dispatch combinations: joining them can create spurious pairings. The chase should be documented using the actual decomposition and tableau symbols in the submitted implementation; do not claim a completed chase run unless the tableau is shown.

Dependency preservation: the stated FDs are enforceable locally in SENDER, SERVICE, ASSIGNMENT, COORDINATOR, COMPLETION_POLICY and SERVICE_BAND. The observation key and foreign keys are enforced in OBSERVATION. Cross-table consequences can require joins or transactional validation if not represented by a direct key/foreign-key constraint.

For independent multivalued facts, model SERVICE_TRANSPORT(ServiceID,TransportMode) and SERVICE_DOCUMENT(ServiceID,DocumentType). If transport modes and document types vary independently for a service, the universal relation ServiceID,TransportMode,DocumentType has independent MVDs ServiceID ↠ TransportMode and ServiceID ↠ DocumentType. Decomposing into the two relations removes the 4NF violation.

For a genuine three-way authorization rule, define AUTHORIZATION(Operator,Item,Channel). If the allowed triples are exactly determined by the three pairwise projections, the join dependency is represented by projections AUTH_OPERATOR_ITEM, AUTH_OPERATOR_CHANNEL and AUTH_ITEM_CHANNEL. The decomposition is justified only when the business rule guarantees that every compatible pairwise combination is authorized; otherwise joining the projections can invent unauthorized triples.

TASK 7 – Relational Algebra and Calculus

Notation: SENDER(S), SERVICE(V), ASSIGNMENT(A), COORDINATOR(C), OBSERVATION(O). The paper does not specify the numeric weights or formula for the 'weighted adjusted index', nor the explicit required-service membership table for each declared group. Those two expressions must be completed using the instructor-provided definitions; inventing them would change the question.

1. Actor 1, wave 4

```
RA: sigma(SenderID=1 AND DispatchWave=4)(OBSERVATION)
TRC: {o | OBSERVATION(o) AND o.SenderID=1 AND o.DispatchWave=4}
DRC: {<s,v,w,c> | OBSERVATION(s,v,w,c) AND s=1 AND w=4}
```

The adjusted-index projection can be added once the weight formula is supplied; the paper's text alone does not define it.

2. Sender records meeting threshold 39 for every required service

```
Let REQUIRED(RouteGroup,ServiceID) list required services.
Good = pi(SenderID,ServiceID)(sigma(Completion>=39)(OBSERVATION))
Bad = REQUIRED joined SENDER minus Good-compatible sender/service pairs.
Answer = pi(SenderID,SenderName)(SENDER) minus senders appearing in Bad.
```

This is a relational-division / NOT EXISTS query. TRC and DRC express the same universal condition with 'for every required service, there exists an observation with Completion >= 39'.

3. Observed service records with no value below 39 in wave 4

```
Eligible = pi(SenderID,ServiceID)(sigma(DispatchWave=4)(OBSERVATION))
Bad = pi(SenderID,ServiceID)(sigma(DispatchWave=4 AND Completion<39)(OBSERVATION))
Answer = Eligible - Bad
```

Starting from Eligible prevents unobserved services from being included vacuously.

4. Coordinator means below observation-weighted hub mean

```
CoordinatorMean = gamma CoordinatorID; AVG(Completion)->Mean(OBSERVATION joined ASSIGNMENT)
HubMean = gamma Hub; AVG(Completion)->HubMean(OBSERVATION joined ASSIGNMENT joined COORDINATOR)
Select coordinator groups where Mean < corresponding HubMean.
```

This uses extended relational algebra (grouping/aggregation). Classical tuple and domain calculus do not directly express AVG; an aggregate extension is required.

5. Senders with sub-threshold observations in at least 3 distinct waves

```
Low = sigma(Completion<39)(OBSERVATION)
Counts = gamma SenderID; COUNT(DISTINCT DispatchWave)->WaveCount(Low)
Answer = pi(SenderID)(sigma(WaveCount>=3)(Counts))
```

TRC/DRC can express the existence of three distinct wave values with three existentially quantified observations. The query must enforce pairwise inequality among their DispatchWave values.

TASK 8 – SQL Implementation and Testing

The following MySQL-oriented DDL implements the BCNF source schema. Use InnoDB so foreign keys and transactions are enforced.

```
CREATE TABLE Sender (
  SenderID INT PRIMARY KEY,
  SenderName VARCHAR(100) NOT NULL,
  RouteGroup VARCHAR(40) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE Service (
  ServiceID INT PRIMARY KEY,
  ServiceCode VARCHAR(30) NOT NULL UNIQUE,
  ServiceName VARCHAR(100) NOT NULL,
  VolumeWeight DECIMAL(8,2) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE Coordinator (
  CoordinatorID INT PRIMARY KEY,
  CoordinatorName VARCHAR(100) NOT NULL,
  Hub VARCHAR(60) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE Assignment (
  SenderID INT NOT NULL,
```

```
  ServiceID INT NOT NULL,
  CoordinatorID INT NOT NULL,
  PRIMARY KEY (SenderID, ServiceID),
  FOREIGN KEY (SenderID) REFERENCES Sender(SenderID) ON DELETE RESTRICT,
  FOREIGN KEY (ServiceID) REFERENCES Service(ServiceID) ON DELETE RESTRICT,
  FOREIGN KEY (CoordinatorID) REFERENCES Coordinator(CoordinatorID) ON DELETE RESTRICT
) ENGINE=InnoDB;
CREATE TABLE CompletionPolicy (
  Completion TINYINT PRIMARY KEY,
  ServiceBand VARCHAR(30) NOT NULL,
  CHECK (Completion BETWEEN 0 AND 100)
) ENGINE=InnoDB;
CREATE TABLE ServiceBand (
  ServiceBand VARCHAR(30) PRIMARY KEY,
  RouteFactor DECIMAL(8,3) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE Observation (
  SenderID INT NOT NULL,
  ServiceID INT NOT NULL,
  DispatchWave INT NOT NULL,
  Completion TINYINT NOT NULL,
  PRIMARY KEY (SenderID, ServiceID, DispatchWave),
  FOREIGN KEY (SenderID, ServiceID)
    REFERENCES Assignment(SenderID, ServiceID) ON DELETE RESTRICT,
  FOREIGN KEY (Completion) REFERENCES CompletionPolicy(Completion) ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Populate CompletionPolicy and ServiceBand with the fixed fictional mapping before loading observations. Add indexes that match measured query predicates, such as Observation(DispatchWave,Completion), Observation(ServiceID,DispatchWave), and Assignment(CoordinatorID). Avoid adding every possible index without measuring its storage and write cost.

Translate the five operational questions using joins, NOT EXISTS/anti-joins, GROUP BY/HAVING and DISTINCT as appropriate. Create role-specific views such as an actor tracking view and an operations view. GRANT SELECT on the relevant views and REVOKE unnecessary privileges as deployment examples. Views expose selected columns; they do not automatically guarantee row-level identity or authorization.

Load the same logical generated data into all three designs using one deterministic seed and identical generation rules. Run anomaly updates inside transactions and ROLLBACK so that the test does not alter the final dataset. Record MySQL version, engine, hardware, indexes, cache state and seed.

TASK 9 – Experimental Results

This task requires execution on the student's local MySQL instance. The assignment paper specifies dataset sizes but supplies no actual benchmark outputs. The following table is a results template; fill it with measured values and do not present estimates as measurements.

Metric	10,000 rows	100,000 rows	600,000 rows	Method
Rows touched: ServiceID=1				SELECT COUNT(*) by design
Rows touched: coordinator fact update				UPDATE trial + ROW_COUNT()
Allocated bytes / pages				InnoDB table/index metrics
Median normal lookup time				Repeated timed runs
Median read-card lookup time				Repeated timed runs

Expanded logical joins				Query plan / SQL structure
Integrity-test result				Capture actual error/result

For each query, warm up consistently, execute multiple repetitions, report the median and state whether the buffer pool was warm or cold. Preserve actual integrity-test messages. Verify that all three designs return identical logical result sets.

TASK 10 – Theory Compared with Experimental Behaviour

The theory predicts that flat-table catalog and coordinator corrections affect multiple observation rows, while the normalized design changes one determinant row. The partial design should reduce sender/service catalog repetition but retain coordinator and policy repetitions. Confirm these predictions using the Task 9 row counts.

The normalized design requires more joins for a sender tracking card; a denormalized read projection can reduce join work. Whether it is faster depends on indexes, cardinality, buffer-pool residency, query plan and hardware. Local timings do not prove that a server can sustain a production request rate. In-memory engines and warm caches can produce results unlike a disk-bound production deployment.

Interpret the actual medians, bytes and row counts here after running the tests. Do not claim a measured speed-up or storage saving until the values have been collected.

TASK 11 – Final Schema Recommendation

Recommend the BCNF schema as the authoritative transactional source because each business fact is stored with its determinant, catalog and coordinator corrections have a single source of truth, and the observation key and assignment scope are explicit. The decomposition is based on the stated dependencies and uses determinant-based joins.

The source schema has additional joins for tracking queries. It also requires careful transactional maintenance of CompletionPolicy and ServiceBand if the fictional policy changes. Derived fields should be computed from the authoritative policy or exposed through a read projection, not independently edited in multiple source rows. Final evidence-based claims about query time and storage must be added from Task 9 measurements.

A view can simplify access and limit visible columns. Actual privileges must be executed and verified separately. Do not describe unexecuted GRANT/REVOKE examples as deployed security controls.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact analysis

The changed workload is 55,000 sender tracking look-ups per hour, with reconciled handover corrections committed every six hours. The average request rate is 55,000 / 3,600 = 15.28 look-ups per second. This is a planning average, not a measured peak rate or a server-throughput guarantee.

The earlier design decision must now weigh read latency and join count more heavily, while corrections remain periodic rather than continuous. A controlled read projection may be acceptable if it is refreshed transactionally after each reconciled correction, carries a source version or refresh timestamp, and has a defined maximum staleness.

B. Evaluate original schema against an indexed read card

The normalized tracking query joins Observation, Assignment, Sender, Service and possibly policy tables. An indexed read card can preassemble the sender-facing tracking fields and answer a lookup with fewer joins. An ordinary SQL view is generally a stored query definition, not necessarily stored results. A transactional materialized read card stores a maintained projection. A permanently duplicated source table creates a second editable source of truth and is the least desirable option.

Compare the normal query and read-card query using the same seeded data, identical filters, indexes and warm/cold cache conditions. Report median times and actual expanded joins. A local benchmark establishes behaviour only for the tested machine, dataset, engine and workload; it does not prove that 55,000 requests/hour will be sustained in production.

C. Adaptation / redesign

Recommended decision: MODIFY. Retain the BCNF schema as the authoritative source and add a controlled, indexed tracking read projection for the high-frequency sender lookup path. Do not replace the normalized source tables with permanently duplicated editable tables.

```
CREATE TABLE SenderTrackingCard (
  SenderID INT NOT NULL,
  ServiceID INT NOT NULL,
  DispatchWave INT NOT NULL,
  SenderName VARCHAR(100) NOT NULL,
  RouteGroup VARCHAR(40) NOT NULL,
  ServiceCode VARCHAR(30) NOT NULL,
  ServiceName VARCHAR(100) NOT NULL,
  CoordinatorName VARCHAR(100) NOT NULL,
  Hub VARCHAR(60) NOT NULL,
  Completion TINYINT NOT NULL,
  ServiceBand VARCHAR(30) NOT NULL,
  RouteFactor DECIMAL(8,3) NOT NULL,
  SourceVersion BIGINT NOT NULL,
  RefreshedAt TIMESTAMP NOT NULL,
  PRIMARY KEY (SenderID, ServiceID, DispatchWave),
  INDEX idx_card_sender_wave (SenderID, DispatchWave)
) ENGINE=InnoDB;
```

Build or refresh the card in the same transaction as the reconciled correction, or publish a versioned snapshot atomically after validation. Validate row counts, foreign-key consistency, policy mapping and result equivalence against the normalized source. Readers should see one consistent snapshot/version, never a partially refreshed card. The six-hour schedule means the design must explicitly define whether the card is refreshed after every correction batch or whether a bounded delay is accepted.

The additional storage equals the measured allocated bytes of the read card and its indexes. The update cost equals the measured rows and bytes written during refresh. These values cannot be calculated from the question paper because the actual generated data, column encodings and index sizes are not supplied.

D. Evidence-based defence

The final decision follows from the declared dependencies: BCNF remains the integrity-preserving source of truth, while the read card is a controlled physical projection introduced for the changed read-heavy workload. The average planning rate is approximately 15.28 requests/second. The final defence must include measured normal/card lookup medians, join counts, allocated bytes, refresh duration, rows refreshed and validation results from this version's local run. Balance those measurements against projection staleness and the risk of inconsistent duplicated data.

Conclusion

The recommended architecture is a BCNF transactional source with an indexed, controlled tracking read projection. This separates authoritative data integrity from read optimization. The recommendation is conditional on successful local measurements and transactional refresh validation; the supplied question paper does not contain benchmark results, so those results must be collected before making quantitative performance claims.